Project Plan StackUnderflow

Agile Software Engineering Capstone

Document field	Team entry
Project title	Financial Management System
Team name	StackUnderflow
Team members	Saw, Zane, Jeff, and Jaron
Student document owner	[Enter the person coordinating updates]
Instructor	Shah, Syed Jawad Hussain
Sponsor or client	Commerce Bank
Version and date	0.1 Draft, November 7th, 2026
Approval status	Draft

How to use this template

Use this document to show how the team will turn the charter, requirements, and backlog into working software during the semester. The plan should make commitments visible without pretending that early estimates will always be exact.

Replace each bracketed prompt with your team's information. Remove unused example rows and all remaining prompts before submission. Keep the document current as the project changes.

Create the first version during project planning. Review it at every sprint boundary and after major changes to scope, capacity, risks, technology, or sponsor expectations.

Revision history

Version	Date	Author	Summary of change	Reviewer or approval
0.1	11/08/2026	Zane,Saw,Jeff,Jaron	Initial draft	[Reviewer]
[Version]	[Date]	[Name]	[What changed]	[Reviewer or decision]

Document outline

Section	Purpose
1 Plan overview	Project goal, MVP, planning approach, priorities, and constraints
2 Release strategy	Major deliverables, milestones, releases, and sprint roadmap
3 Backlog and work breakdown	Prioritized work, WBS, and work package details
4 Estimation and capacity	Effort, duration, assumptions, and available team time
5 Schedule and dependencies	Task relationships, dates, critical work, and resource balancing
6 Sprint operating plan	Sprint cadence, iteration plan, Definition of Ready, and Definition of Done
7 Roles and responsibilities	Ownership, backup coverage, and decision authority
8 Communication plan	Information needs, channels, timing, and escalation
9 Quality plan	Reviews, testing, acceptance, defects, and documentation
10 Risks and issues	Possible problems, active problems, responses, and owners
11 Change and configuration control	How the team manages scope, code, documents, and releases
12 Progress tracking	Measures, reports, forecasts, and plan updates
13 Tools and environments	Shared tools, access, and development or test environments
14 Review and submission	Approvals and final completeness check

1 Plan overview

This section gives the reader a quick picture of what the team plans to deliver and how the team will organize the work.

1.1 Product goal and minimum viable product

Our Financial Management Application will help everyday users save money to reach their personal financial goals. This will include an emergency savings goal, personalized savings goals, personalized wishlists, and feedback on how to save based on the users budget and previous savings deposits. This application will include registration, login, profile settings, a budget page for user reported expenses/income, a dashboard, automated tests, an accounts page to manage savings/wishlists, and a financial page to show transaction history.

1.2 Planning approach and planning horizon

The team will use two-week sprints. Outlining goals and outcomes for each sprint. These goals and outcomes should be completed before the following sprint but adopted into the next sprint if not completed.

1.3 Priorities constraints and assumptions

Type	Statement	Why it matters	How the team will respond
Priority	Fully functional core aspects of application before applying stretch goals	Stretch goals are meaningless without core app functionality	Ensure goals of core functionality has been reached before moving onto stretch goals
Constraint	Fixed deadline	Project is due at deadline, cannot work past deadline	Ensure productivity and efficiency in the team in order to complete the project before the deadline
Assumption	[Condition believed to be true]	[Effect if wrong]	[How and when the team will check]

2 Release strategy

A release plan shows the overall path to the final product. It stays less detailed than an individual sprint plan.

2.1 Major deliverables and milestones

ID	Deliverable or milestone	Evidence	Target date or sprint	Owner	Reviewer
M-01	Architecture candidate reviewed	Architecture document and ADRs	Sprint 1	Architecture lead	Instructor

ID	Deliverable or milestone	Evidence	Target date or sprint	Owner	Reviewer
M-02	Midsemester check in	Working demo and feedback notes	10/26/2026	Team lead	Commerce Bank
M-03	[Deliverable or milestone]	[What proves completion]	[Date or sprint]	[Name]	[Name or role]

2.2 Release and sprint roadmap

What to write: *List one clear goal and one demonstrable outcome for each sprint. Include dependencies and review evidence. Avoid using the roadmap as a list of individual assignments.*

Sprint	Sprint goal	Planned demonstrable outcome	Main dependencies	Review evidence
Sprint 0	Align and prepare	Charter, initial requirements, architecture sketch, repository, board, and test strategy	Sponsor brief and team access	Reviewed documents and working repository
Sprint 1	Front-end UI	The conceptual web-app functions from a user perspective. Dummy information, no real implementation	Initial architecture and frontend	Live demo
Sprint 2	Backend Setup	The database and back-end logic foundation has been implemented	Initial architecture, Database, backend	Demo
Sprint 3	Further Development of Backend/Frontend	Frontend has consistent design throughout pages. Backend logic for core app features has been implemented	Architecture, Database, Backend, frontend, UI design	Demo
Sprint 4	Connection between Frontend and backend	Frontend now communicates with backend/database	ASP.NET,Database, Backend, frontend	Demo/Test
Sprint 5	Real functionality fully implemented	No dummy data in frontend. All information is pulled from the database through the backend. Frontend/backend functionality fully working	ASP.NET,Database, Backend, frontend	Test
Sprint 6	AI implementation and Stretch goals	AI financial advising and chosen 2 stretch goals are implemented and fully functional		Demo/Test
Sprint 7	Bug fixing	All crucial bugs have been patched. Deployment is functional		Test

3 Backlog and work breakdown

3.1 Prioritized backlog summary

What to write: *Summarize the important epics or stories and identify what belongs to the MVP. Keep detailed acceptance criteria in the requirements document or backlog tool.*

Backlog ID	Epic story or deliverable	Priority	Estimate	Target sprint	Status
US-01	As a registered user, I can report my budget through my income/expenses, so that I can get a personalized experience	Must	8 points	Sprint 2	Active
US-02	As a registered user, I can deposit money into my account(s), so that I can work towards my saving goals	Must	10 points	Sprint 3	Ready
US-03	As a registered user, I can get feedback on my savings history, so that I can continue to save towards my goals more efficiently	Must	6 points	Sprint 5	Proposed
US-04	As a registered user, I can go into my profile settings, so that I can change different aspects of my profile	Should	2 points	Sprint 6	Proposed

3.2 Work breakdown structure

What to write: *Break major deliverables into summary tasks and schedulable work packages. Use a numbered outline or a WBS diagram. Every leaf item should produce a clear result.*

Example: 1.0 User identity; 1.1 Confirm login requirements; 1.2 Design user table; 1.3 Implement registration; 1.4 Implement login; 1.5 Test authorization; 1.6 Demonstrate completed identity flow.

3.3 Work package details

WBS ID	Work package	Owner	Output and completion test	Effort	Duration	Predecessor
1.3	Example: Implement registration API	Backend developer	Valid user stored with hashed password; tests pass	6 hours	3 days	1.2
[ID]	[Specific task]	[Name]	[Deliverable and acceptance condition]	[Person-hours]	[Calendar time]	[ID or none]

4 Estimation and capacity

4.1 Estimation method and assumptions

What to write: *State how the team estimates work, when estimates are revised, and what completion includes. Name assumptions such as team experience, available environments, and review time.*

Example: The team will use planning poker for story points and person-hours for work packages. Estimates include coding, review, testing, integration, and documentation. The team will compare estimates with actual effort after every sprint.

4.2 Effort and duration

Effort means labor time. Duration means calendar time. A task may require two hours of effort but three days of duration while the team waits for a review or external decision.

Task	Effort estimate	Duration estimate	Reason for difference	Confidence
Example: Sponsor review	1 person-hour	4 calendar days	The team must wait for sponsor availability	Medium
[Task]	[Person-hours]	[Calendar time]	[Reason]	[Low / Medium / High]

4.3 Team capacity

What to write: *Estimate realistic availability after classes, jobs, holidays, planned absences, meetings, and normal interruptions. Do not assume that every available hour becomes feature work.*

Team member	Role	Hours available per week	Planned absence	Focus factor	Usable sprint capacity	Backup
Example: Alex	Front-end lead	8	None	75%	12 hours in a two-week sprint	Sam
[Name]	[Role]	[Hours]	[Dates]	[Percent]	[Hours]	[Name]

5 Schedule and dependencies

5.1 Task dependencies

From task	To task	Dependency type	Explanation	Risk if delayed
Database design	Registration API	Finish to start	The API depends on an agreed user schema	Identity work slips
UI implementation	UI test preparation	Start to start	Test design can begin after the UI work begins	Late test coverage
[Task A]	[Task B]	[Finish-start / Finish-finish / Start-start / Start-finish]	[Why]	[Effect]

5.2 Schedule

What to write: *Insert a Gantt chart, sprint calendar, or linked schedule. Show start and finish dates, dependencies, milestones, reviews, and releases. Keep the schedule consistent with the roadmap and team capacity.*

[Insert schedule or link to the current schedule here]

5.3 Critical work and float

What to write: *Identify tasks that directly control a major deadline. Record any available slack and the response if critical work slips.*

Critical task or path	Deadline controlled	Available slack	Early warning	Response
Example: Identity schema to authorization tests	Midsemester demo	2 days	Login flow fails by sprint midpoint	Pair on minimal supported flow and defer optional account features
[Task or path]	[Milestone]	[Time]	[Trigger]	[Response]

5.4 Resource balancing

What to write: *Describe overloaded people or shared resources and how the team will move, split, pair, or defer work.*

6 Sprint operating plan

6.1 Sprint cadence

Event	Purpose	Participants	Timing and timebox	Output
Sprint planning	Choose a sprint goal and work that fits capacity	Whole team	First day, 60 minutes	Sprint goal and sprint backlog
Standup	Share progress and blockers	Whole team	[Schedule], 10-15 minutes	Updated board and follow-up actions
Review or demo	Show working evidence and obtain feedback	Team and reviewers	Last day, [timebox]	Feedback and acceptance decisions
Retrospective	Improve the team's way of working	Student team	After review, [timebox]	One or two improvement actions

6.2 Sprint plan

Sprint field	Team entry
Sprint number and dates	[Enter sprint and dates]
Sprint goal	[One outcome the team can evaluate at the review]
Committed stories or deliverables	[IDs and short names]
Capacity and known absences	[Available person-hours and constraints]
Main dependencies	[People, decisions, environments, or earlier work]
Review evidence	[Demo, test results, document, or approval]
Plan adjustment rule	[How the team will protect the sprint goal if work changes]

6.3 Definition of Ready and Definition of Done

Definition	Required conditions
Ready	Example: user value and acceptance criteria are clear; dependencies are identified; the item is small enough for one sprint; the team can test it
Done	Example: acceptance criteria pass; code is reviewed and merged; relevant tests pass; defects are recorded; documentation and traceability are updated; the increment can be demonstrated
Team Ready definition	[Enter the team's conditions]
Team Done definition	[Enter the team's conditions]

7 Roles and responsibilities

Role or person	Main responsibilities	Decision authority	Backup	Expected evidence
Project coordinator	Maintains plan, facilitates ceremonies, tracks impediments	Routine plan updates within approved scope	[Name]	Current plan and decision log
[Role or name]	[Responsibilities]	[Decision area]	[Name]	[Output]

8 Communication plan

What to write: *Show what information each stakeholder needs, who creates it, how it is delivered, and when. Important decisions should not exist only in chat messages.*

Information	Creator or source	Audience	Method or location	Frequency or trigger	Owner
Sprint status	Project board and team	Instructor and sponsor	Short written update plus demo	End of each sprint	Coordinator

Information	Creator or source	Audience	Method or location	Frequency or trigger	Owner
Technical decision	Team discussion	Developers and testers	Decision log or ADR	When a significant decision occurs	Architecture lead
[Information]	[Source]	[Audience]	[Channel]	[Timing]	[Name]

8.1 Team communication rules

What to write: *State the primary channel, expected response time, meeting etiquette, respectful communication expectations, decision recording, and escalation route.*

---

9 Quality plan

Quality activity	When it occurs	Owner	Entry condition	Completion evidence
Code review	Before merge	Peer reviewer	Automated checks pass	Approved pull request
Unit testing	During implementation	Story owner	Testable behavior identified	Passing tests in CI
Integration or UI testing	Before sprint review or release	Test owner	Increment deployed to test environment	Recorded results and defects
[Activity]	[Timing]	[Name or role]	[Condition]	[Evidence]

9.1 Defect handling

What to write: *Define severity levels, who triages defects, which defects block a sprint or release, and how unresolved defects are documented.*

---

10 Risks and issues

10.1 Risk register

Risk ID	Possible problem	Probability	Impact	Trigger	Mitigation and contingency	Owner
R-01	Authentication integration takes longer than planned	Medium	High	Login flow does not work by sprint midpoint	Pair on a minimal library-supported flow; defer optional identity features	Backend lead
R-__	[Risk]	[L/M/H]	[L/M/H]	[Early warning]	[Prevention and backup response]	[Name]

10.2 Issue log

Issue ID	Current problem	Effect	Action	Owner	Due date	Status
I-01	Example: Test database access unavailable	Integration testing blocked	Request access and prepare local fallback	[Name]	[Date]	Open
I-__	[Issue]	[Effect]	[Next action]	[Name]	[Date]	[Open / Closed]

11 Change and configuration control

11.1 Scope and plan change process

Step	Team procedure
1 Request	Record the requested change, reason, requester, and date
2 Analyze	Estimate value, effort, schedule, risk, quality, and document effects
3 Decide	[Name who may approve routine and major changes]
4 Update	Revise backlog, requirements, plan, architecture, tests, and decision log as needed
5 Communicate	Tell affected stakeholders what changed and why

11.2 Configuration management

What to write: *Describe repository structure, branching, pull requests, version tags, document storage, baselines, release naming, and access rules.*

12 Progress tracking

Measure	How calculated or judged	Source	Frequency	Decision it supports
Sprint goal result	Met, partially met, or not met with explanation	Sprint review	Each sprint	Adjust scope or process
Forecast	Expected completion based on remaining work and observed capacity	Backlog and capacity	Each sprint	Move or split work
Test and defect status	Required tests passing and open defects by severity	CI and issue tracker	At least weekly	Release readiness
[Measure]	[Method]	[Source]	[Timing]	[Decision]

12.1 Status report format

What to write: *Use a short report: completed since last report, next work, blockers, risk changes, quality status, forecast, and decisions needed.*

---

13 Tools and environments

Purpose	Tool or environment	Location	Access owner	Use and backup notes
Source control	Github	https://github.com/ZaneD03/stackunderflow	Zane	Master Branch
Project board	[Tool]	[Board link]	[Name]	[Status rules]
Development	[Local or shared setup]	[Setup guide]	[Name]	[Configuration notes]
Test or demo	[Environment]	[URL or location]	[Name]	[Deployment and reset notes]

14 Review and submission

14.1 Review decisions

Reviewer	Role	Decision	Conditions or comments	Date
[Name]	Project sponsor	[Approve / Revise]	[Comments]	[Date]
Shah, Syed Jawad Hussain	Instructor	[Approve / Revise]	[Comments]	[Date]
[Name]	Student project lead	[Team commitment]	[Comments]	[Date]

14.2 Final submission checklist

[ ] The plan agrees with the charter, requirements, architecture, and course calendar.

[ ] Mandatory work and the MVP appear before optional or stretch work.

[ ] Every sprint has a clear goal and demonstrable outcome.

[ ] The WBS reaches schedulable work packages with owners and completion tests.

[ ] Effort, duration, dependencies, capacity, and planned absences are realistic.

[ ] Testing, integration, documentation, reviews, and defect work appear in the plan.

[ ] Communication, risk, change, configuration, and progress practices are clear.

[ ] The plan records what changed after the latest sprint review or retrospective.

[ ] All bracketed prompts and unused example rows have been removed.

[ ] The version, date, author list, and approval decisions are current.

Recommended filename: TeamName_ProjectPlan_v1.0.docx